Project 2: CUDA blob detection

Tommaso Ciccotti
7196985
tommaso.ciccotti@edu.unifi.it

Abstract

This project develops a multi-scale blob detector based on the Difference of Gaussians. The implementation includes a sequential CPU pipeline, a 12-thread OpenMP pipeline and a CUDA pipeline tested with 11 block layouts. Five Gaussian levels produce four DoG maps which are searched for extrema. Every backend uses the same non-maximum-suppression radius of six pixels and the same 13×13 *window. The benchmark covers four resolutions and four thresholds with five repetitions for each configuration. CPU and OpenMP match the sequential reference in every test while CUDA matches exactly in 110 of 176 cases. The remaining differences affect between one and 24 binary pixels and are identical for every block layout at the same resolution and threshold. OpenMP speedup ranges from* $4.4051\times$ *to* $5.5379\times$ *while the highest verified CUDA speedup is* $192.4353\times$*. Resolution is the main source of runtime growth while threshold changes have a smaller effect because Gaussian filtering and DoG construction process the complete image. The report analyses backend consistency, threshold sensitivity, resolution scaling, block geometry, timing stability and the remaining numerical differences between CPU and GPU execution.*

1. Introduction

Blob detection is a common feature-extraction operation in digital image processing and computer vision. It identifies localized regions whose intensity differs from the surrounding neighbourhood across one or more spatial scales. These regions can support object tracking, keypoint registration and multi-scale image analysis.

A Difference of Gaussians pipeline creates several blurred versions of the same image. Adjacent levels are subtracted and the resulting maps are searched for local extrema. With a fixed number of levels the workload grows with the number of pixels. Doubling both image dimensions therefore produces about four times as much work.

Most output pixels can be processed independently after the required input layer has been generated. This makes the pipeline suitable for OpenMP on the CPU and CUDA on the GPU.

The RGB channels are converted on the CPU to normalized luminance values in the interval $[0, 1]$:

$$Y = \frac{0.299 \cdot R + 0.587 \cdot G + 0.114 \cdot B}{255.0} \quad (1)$$

The objective is to compare the sequential, OpenMP and CUDA paths across several resolutions, thresholds and block layouts. The analysis considers execution time, scaling, stability and output consistency.

2. Implementation

The application divides the blob-detection process into a fixed sequence of scale-space generation, Difference of Gaussians computation, three-dimensional extrema detection and spatial non-maximum suppression.

2.1. Pipeline

- *Scale-space generation*: Five Gaussian layers are generated using

$$\sigma_i = \sigma_0 k^i, \quad (2)$$

where $\sigma_0 = 1.25$, $k = 1.414$ and $i \in [0, 4]$. The nominal scale values are approximately

1.25, 1.77, 2.50, 3.53, and 5.00. Each two-dimensional Gaussian filter is implemented as a horizontal pass followed by a vertical pass. The radius is

$$r = \lceil 3\sigma_i \rceil, \tag{3}$$

and boundary coordinates are clamped explicitly to the image limits. For the horizontal pass:

$$I_i(x, y) = \frac{\sum_{m=-r}^{r} G(m, \sigma_i) Y(x + m, y)}{\sum_{m=-r}^{r} G(m, \sigma_i)}. \tag{4}$$

- *Difference of Gaussians*: Four DoG maps are obtained by subtracting adjacent Gaussian layers:

$$D_i(x, y) = L_{i+1}(x, y) - L_i(x, y). \tag{5}$$

Here $i \in [0, 3]$. The two internal DoG layers, D_1 and D_2, can be compared with the preceding and following scale layers during the extrema search.

- *Three-dimensional extrema detection*: For each valid image coordinate and internal DoG layer the implementation first checks whether

$$|D_i(x, y)| \geq \tau, \tag{6}$$

where τ is the detection threshold. A surviving value is compared with the 26 neighbours in the surrounding $3 \times 3 \times 3$ space-scale neighbourhood. Equality does not invalidate a candidate so the implementation detects non-strict local maxima or minima.

- *Non-maximum suppression*: The sequential, OpenMP and CUDA paths use a radius of six pixels. Each candidate is therefore compared inside the same 13×13 spatial window. All implementations preserve the absolute DoG response in the temporary extrema map and use that strength during suppression. A surviving blob location is stored as 1.0 while all other locations are 0.0.

2.2. CPU paths

The sequential implementation performs every stage on the CPU. The OpenMP path uses 12 threads and parallelizes the Gaussian passes, DoG computation, extrema search and non-maximum suppression with `schedule(static)`. The horizontal and vertical Gaussian passes are enclosed in the same parallel region. The implicit barrier after the horizontal loop completes the temporary image before the vertical pass starts and each iteration writes to an independent output position.

The extrema and NMS loops also assign one output coordinate to each iteration.

2.3. CUDA path

The CUDA implementation maps one image coordinate to each GPU thread. Eleven block layouts are tested:

$$8 \times 8,\ 16 \times 16,\ 32 \times 32,\ 64 \times 16,\ 16 \times 64,$$
$$128 \times 8,\ 8 \times 128,\ 256 \times 4,\ 4 \times 256,$$
$$1024 \times 1,\ 1 \times 1024.$$

The horizontal blur, vertical blur, DoG subtraction, extrema detection and non-maximum suppression kernels run in the default stream. Each invocation allocates the device buffers and copies the input. It then launches the kernels, copies the output to the host and frees the buffers. CUDA times include allocation, transfers, kernel execution, synchronization and deallocation.

2.4. Data dependencies

The pipeline contains dependencies between stages but each stage exposes a large set of independent pixel operations. A vertical Gaussian pass cannot start before the horizontal pass has completed because it reads the temporary image and the DoG stage cannot begin before the required Gaussian levels exist. Extrema detection requires three adjacent DoG layers and NMS requires the complete extrema-strength map.

These dependencies are handled with stage boundaries, the OpenMP barriers allow this separation while CUDA launches the kernels in the

default stream so they execute in issue order. A device synchronization is performed before the final result is used on the host.

This structure avoids atomic operation, critical sections and makes the result independent of the order in which pixels are scheduled.

2.5. Computational cost

The separable Gaussian filter replaces a two-dimensional kernel with one horizontal and one vertical pass. For a radius r each level requires approximately

$$2WH(2r+1) \tag{7}$$

weighted samples instead of $WH(2r+1)^2$. The five scale levels use different radii because the Gaussian standard deviation increases with the scale index.

DoG subtraction has a lower arithmetic cost, it requires one subtraction for each pixel and each adjacent scale pair. Extrema detection is more irregular, values below the threshold leave the stage early while surviving candidates may inspect up to 26 neighbours. NMS is also candidate dependent as a non-zero extrema response can inspect the surrounding 13×13 window.

This cost distribution explains the benchmark trends. Resolution affects every stage and multiplies the complete amount of work while threshold changes only the later candidate dependent stages. Its effect on runtime is therefore smaller than its effect on the number of detected features.

2.6. Memory behaviour

All images are stored in row-major order. The horizontal Gaussian passes read adjacent elements and produce contiguous accesses for neighbouring workers. Vertical passes read values separated by one row, they provide less spatial locality and place greater pressure on caches or global-memory transactions.

CUDA block shape changes how a warp maps onto this layout. A block with a useful horizontal extent assigns consecutive threads to nearby elements in the same row. Instead a fully vertical layout sends consecutive threads to addresses separated by the image width. This difference does not change the mathematical result but it can change memory efficiency.

3. Setup

The benchmark evaluates four image sizes, four thresholds, one OpenMP thread count and 11 CUDA block layouts.

3.1. Hardware

The benchmark system uses an AMD Ryzen 5 3600X with 6 physical cores and 12 logical threads, 16 GB of DDR4 memory and an NVIDIA GeForce RTX 2070 SUPER with 8 GB of video memory. The operating system is Windows 10 .

3.2. Build

The project is built using CMake version 3.24 or higher with both C++ and CUDA standards set to C++17. If no build type is specified CMake selects Release mode. GNU and Clang use `-O3 -march=native` with OpenMP support for host code while Microsoft Visual C++ uses `/O2 /Oi /Ot` and `/openmp`. CUDA code is compiled with `-O3` and the target architecture is set to 75. These optimization levels allow compiler auto-vectorization where applicable.

3.3. Benchmark

- *Resolutions*: 512×512, 1024×1024, 2048×2048 and 4096×4096 pixels. The larger images are generated from the base image with nearest-neighbour sampling.
- *Repetitions and timing*: Each configuration is executed five times. Mean, minimum, maximum and standard deviation are computed with host-side `std::chrono` measurements. CPU times include the allocation of intermediate vectors. CUDA times include device allocation, transfers, kernel execution, synchronization and deallocation. No CUDA warm-up is performed.

- *Verification*: The final output is compared pixel by pixel with the sequential reference using an absolute tolerance of 0.01. The CSV records the match result, the number of different pixels and the maximum absolute difference. Verification is performed outside the timed region.
- *Execution conditions*: On Windows `SetThreadExecutionState` prevents the system from entering sleep mode during the benchmark.

3.4. Parameters

The threshold values are 0.0005, 0.0050, 0.0170 and 0.0500. The range covers two orders of magnitude, higher thresholds reject more values before the 26-neighbour comparison.

The OpenMP path uses 12 threads while CUDA is tested with 11 block shapes containing between 64 and 1024 threads. The layouts include row-oriented, column-oriented and symmetric configurations.

3.5. Metrics

The benchmark uses five repetitions for every configuration. Let T_i be the end-to-end runtime of repetition i. The average execution time is

$$T_{\text{avg}} = \frac{1}{R}\sum_{i=1}^{R} T_i, \tag{8}$$

Minimum and maximum times are

$$T_{\min} = \min(T_1, \ldots, T_R), \tag{9}$$

$$T_{\max} = \max(T_1, \ldots, T_R). \tag{10}$$

Runtime variation is measured with the population standard deviation

$$\sigma_T = \sqrt{\frac{1}{R}\sum_{i=1}^{R} \left(T_i - T_{\text{avg}}\right)^2} \tag{11}$$

and the coefficient of variation

$$CV = \frac{\sigma_T}{T_{\text{avg}}} \times 100. \tag{12}$$

The standard deviation measures the dispersion of the repeated runtimes around their average value. A low value indicates stable execution times while a higher value indicates greater runtime variability. The coefficient of variation normalizes this dispersion with respect to the average runtime and expresses it as a percentage. The speedup compares a backend with the sequential execution at the same resolution and threshold:

$$S = \frac{T_{\text{seq}}}{T_{\text{backend}}}. \tag{13}$$

OpenMP parallel efficiency uses the fixed 12-thread configuration:

$$E_{\text{OpenMP}} = \frac{S_{\text{OpenMP}}}{12}. \tag{14}$$

CUDA block layouts are not evaluated with a thread efficiency formula because the number of logical CUDA threads is not comparable to the number of OpenMP CPU threads. Changing the block shape also changes the number of blocks and the memory-access pattern. CUDA layout efficiency is therefore measured against the fastest layout for the same resolution and threshold:

$$E_{\text{CUDA,layout}}(b) = \frac{T_{\text{CUDA,best}}}{T_{\text{CUDA}}(b)}. \tag{15}$$

This value measures how close one block layout is to the best CUDA layout for the same workload and is not directly comparable with OpenMP parallel efficiency.

The normalized processing cost is expressed in nanoseconds per pixel:

$$C_{\text{px}} = \frac{T_{\text{avg}} \times 10^6}{WH}. \tag{16}$$

Output consistency is evaluated separately from performance, the final binary map is compared with the sequential reference.

4. Results analysis

4.1. Backend consistency

The sequential and OpenMP implementations produce the same binary output at every resolu-

tion and threshold. CUDA also matches the sequential reference at 512×512, differences appear at larger resolutions when the threshold is low.

Table 1 shows the CUDA results for each resolution and threshold, *match* means that the complete binary output is identical to the sequential reference while the other entries report the number of different output pixels. Every CUDA block layout produces the same result for a fixed resolution and threshold.

Table 1. CUDA output consistency against the sequential reference.
Legend: Match = exact output; n px = n different binary pixels.

Resolution	$\tau =$ 0.0005	$\tau =$ 0.0050	$\tau =$ 0.0170	$\tau =$ 0.0500
512^2	Match	Match	Match	Match
1024^2	1 px	Match	Match	Match
2048^2	1 px	Match	Match	Match
4096^2	24 px	16 px	5 px	1 px

Higher thresholds improve CUDA output consistency. The result is exact for every threshold at 512^2. At 1024^2 and 2048^2 only the lowest threshold changes one pixel. At 4096^2 the difference becomes larger but falls from 24 pixels to one as the threshold increases.

Block geometry changes the runtime but not the final output. The remaining difference is therefore linked to numerical operations before thresholding and NMS. A response close to a decision boundary can be represented slightly differently by the CPU and CUDA and can change the state of a small number of output pixels.

4.2. Threshold effect

The threshold controls how many DoG responses continue to the complete 26-neighbour comparison and how many non-zero extrema enter NMS. A higher threshold rejects weak responses earlier and reduce noise but does not reduce the cost of the Gaussian stages.

Table 2 reports the mean runtime at the lowest and highest thresholds. The last column shows the percentage change from the first time to the second:

$$\Delta T = \frac{T_{0.0500} - T_{0.0005}}{T_{0.0005}} \times 100. \qquad (17)$$

A negative value means that execution is faster at threshold 0.0500. CUDA uses the fixed 32×32 block layout.

Table 2. Mean runtime at the lowest and highest thresholds.
Legend: Times are in ms; ΔT is the change from $\tau = 0.0005$ to $\tau = 0.0500$.

Backend	**Resolution**	$T_{0.0005}$	$T_{0.0500}$	ΔT
Sequential	512^2	284.88	257.09	-9.8%
Sequential	1024^2	1111.91	1020.80	-8.2%
Sequential	2048^2	4332.35	4088.45	-5.6%
Sequential	4096^2	17356.12	15970.91	-8.0%
OpenMP 12T	512^2	63.81	57.96	-9.2%
OpenMP 12T	1024^2	252.41	204.80	-18.9%
OpenMP 12T	2048^2	848.14	834.69	-1.6%
OpenMP 12T	4096^2	3225.38	2994.66	-7.2%
CUDA 32×32	512^2	2.44	2.50	$+2.1\%$
CUDA 32×32	1024^2	7.34	6.92	-5.7%
CUDA 32×32	2048^2	25.14	22.68	-9.8%
CUDA 32×32	4096^2	82.55	72.63	-12.0%

The sequential runtime decreases at every resolution with the greater threshold. The reduction remains between 5.6% and 9.8% as the threshold affects the candidate-dependent stages but the Gaussian passes still dominate the complete CPU pipeline.

OpenMP follows the same general tendency. The reduction is strongest at 1024^2 and weakest at 2048^2, the variation is less regular because the runtime includes several parallel stages and each mean is calculated from five repetitions.

CUDA shows little threshold sensitivity at 512^2. The increase from 2.44 ms to 2.50 ms is small and fixed overhead has a greater influence on this workload. At larger resolutions the runtime decreases as the threshold rises and the reduction reaches 12.0% at 4096^2 because more weak candidates are rejected before the complete comparison and NMS stages.

4.3. Block shape

The fastest layout changes with the threshold. The 16×16 layout leads at 0.0005 and 0.0170 while the 256×4 layout leads at 0.0050 and

Table 3. CUDA block-shape comparison at 512×512.
Legend: Layout = CUDA block shape; τ = threshold; T_{avg} in ms; S = verified speedup.

Layout/metric	$\tau =$ 0.0005	$\tau =$ 0.0050	$\tau =$ 0.0170	$\tau =$ 0.0500
8×8 T_{avg}	34.7695	2.7319	2.6442	2.6183
8×8 S	8.19×	100.26×	98.53×	98.19×
16×16 T_{avg}	2.4282	2.6202	2.5941	2.4758
16×16 S	117.32×	104.53×	100.43×	103.84×
32×32 T_{avg}	2.4433	2.6943	2.7152	2.4950
32×32 S	116.60×	101.66×	95.95×	103.04×
64×16 T_{avg}	2.4864	2.7410	2.7538	2.5754
64×16 S	114.58×	99.92×	94.61×	99.82×
16×64 T_{avg}	2.5787	2.6951	2.8424	2.6500
16×64 S	110.48×	101.63×	91.66×	97.01×
128×8 T_{avg}	2.7111	2.8105	2.7611	2.6219
128×8 S	105.08×	97.46×	94.36×	98.05×
8×128 T_{avg}	2.5993	2.5799	2.5961	2.5164
8×128 S	109.60×	106.17×	100.35×	102.16×
256×4 T_{avg}	2.6751	2.4666	2.6359	2.3491
256×4 S	106.50×	111.04×	98.84×	109.44×
4×256 T_{avg}	2.8222	2.7027	2.7990	2.4581
4×256 S	100.94×	101.34×	93.08×	104.59×
1024×1 T_{avg}	2.8119	2.6724	2.7349	2.4307
1024×1 S	101.31×	102.49×	95.26×	105.76×
1×1024 T_{avg}	4.7628	4.4644	4.2666	4.1932
1×1024 S	59.81×	61.35×	61.06×	61.31×

0.0500. The strongest layouts remain close in absolute time.

The fully vertical 1×1024 layout is consistently slower as layouts with a useful horizontal extent access more adjacent row-major elements.

The 8×8 result at the lowest threshold contains one large outlier. Its minimum is comparable with the other layouts while one long run raises the mean and standard deviation, this pattern appears across several resolutions. The other layouts present similar times but the symmetrical and horizontal configurations show higher maximum speedups.

4.4. Resolution scaling

A speedup of $4\times$ means that the backend completes the same test in one fourth of the sequential time.

The sequential runtime increases by about $60.9\times$ from the smallest to the largest resolution

Table 4. Resolution scaling at threshold 0.0005 using a fixed CUDA 32×32 layout.
Legend: Runtimes are end-to-end means in ms; speedup is calculated against the sequential runtime.

Metric/backend	512^2	1024^2	2048^2	4096^2
Pixel count	262,144	1,048,576	4,194,304	16,777,216
CPU sequential	284.8817	1111.9064	4332.3549	17356.1172
OpenMP, 12 threads	63.8132	252.4120	848.1413	3225.3750
OpenMP speedup	4.46×	4.41×	5.11×	5.38×
CUDA 32×32	2.4433	7.3368	25.1362	82.5516
CUDA speedup	116.60×	151.55×	172.36×	210.25×
CUDA match	Yes	No	No	No
CUDA different pixels	0	1	1	24

while the pixel count increases by $64\times$. The CPU workload therefore remains close to linear in the number of pixels.

OpenMP increases by about $50.5\times$ over the same range. Its speedup improves on the larger images because more work is available for each parallel region and fixed overhead has a lower relative weight.

CUDA increases by about $33.8\times$. Allocation and launch costs form a larger part of the small-image runtime but they are amortized as the image grows. CUDA speedup therefore rises from $116.60\times$ at 512^2 to $210.25\times$ at 4096^2.

4.5. OpenMP efficiency and CUDA layout efficiency

OpenMP and CUDA use different efficiency measures. OpenMP efficiency describes the CPU thread scaling while CUDA layout efficiency describes how close one block shape is to the fastest CUDA setup for the same test.

Table 5 reports OpenMP efficiency with 12 threads.

Table 5. OpenMP parallel efficiency with 12 threads.
Legend: $E_{\text{OpenMP}} = S_{\text{OpenMP}}/12$; values are percentages.

Resolution	$\tau = 0.0005$	$\tau = 0.0050$	$\tau = 0.0170$	$\tau = 0.0500$
512^2	37.2%	37.1%	37.4%	37.0%
1024^2	36.7%	38.5%	42.4%	41.5%
2048^2	42.6%	43.6%	41.8%	40.8%
4096^2	44.8%	45.2%	46.1%	44.4%

Efficiency remains close to 37% at 512^2 and it increases with resolution and reaches 46.1% at 4096^2 and threshold 0.0170. Larger images

provide more work between the synchronization points and reduce the relative effect of parallel overhead.

The threshold produces a smaller variation than resolution.

Table 6 compares all CUDA setups. Mean efficiency is calculated across the 16 resolution-threshold tests.

Table 6. CUDA layout efficiency across all resolution-threshold tests.
Legend: $E_{\mathrm{CUDA,layout}} = T_{\mathrm{best}}/T_b$; values are percentages.

Layout	Mean efficiency	Minimum efficiency	Fastest tests
8×8	43.4%	7.0%	0/16
16×16	97.0%	89.8%	6/16
32×32	92.5%	82.9%	0/16
64×16	93.7%	88.0%	1/16
16×64	93.0%	88.6%	0/16
128×8	95.4%	87.8%	2/16
8×128	94.9%	89.1%	1/16
256×4	95.5%	88.1%	3/16
4×256	87.4%	81.0%	0/16
1024×1	96.0%	86.4%	3/16
1×1024	49.7%	40.6%	0/16

The 16×16 layout has the highest mean efficiency at 97.0% and produces the fastest time in six tests. The 1024×1 and 256×4 layouts also remain above 95% on average.

The 8×8 layout falls to 43.4% because several measurements contain a large runtime outlier while the 1×1024 layout reaches 49.7%. Its vertical shape gives consecutive threads row-separated memory addresses and remains slower across the tested workloads.

The strongest layouts remain close to the fastest CUDA time. The main efficiency losses appear in the smallest symmetrical and the fully vertical layouts rather than in the horizontal or bigger symmetrical configurations.

4.6. Timing stability

CV allows configurations with different runtime scales to be compared.

The sequential backend is the most stable. Its mean CV is 1.47% and its maximum is 5.01%. OpenMP has greater variation because thread scheduling and synchronization affect each repetition, its mean CV is 8.32%.

Table 7. Timing stability grouped by backend.
Legend: Values are CV percentages across all configurations of the backend.

Backend	Mean CV	Median CV	Minimum CV	Maximum CV
Sequential	1.47%	1.19%	0.16%	5.01%
OpenMP 12T	8.32%	6.16%	1.29%	18.37%
CUDA, all layouts	11.83%	4.51%	0.29%	185.23%
CUDA, without 8×8	5.16%	4.03%	0.29%	17.13%

The CUDA mean is increased by the 8×8 layout as across all layouts the mean CV is 11.83% while the median is 4.51%. Removing 8×8 reduces the mean to 5.16% and the maximum to 17.13%. The difference between mean and median therefore comes from one unstable layout rather than from general CUDA behaviour.

Table 8. Timing stability of the CUDA block layouts.
Legend: Mean, median and maximum CV.

Layout	Mean CV	Median CV	Maximum CV
8×8	78.55%	77.36%	185.23%
16×16	5.55%	5.92%	10.70%
32×32	4.84%	4.71%	12.75%
64×16	5.40%	4.51%	16.30%
16×64	5.38%	4.19%	11.63%
128×8	5.33%	3.09%	16.47%
8×128	5.41%	3.89%	13.59%
256×4	6.22%	4.43%	14.61%
4×256	5.05%	3.01%	17.13%
1024×1	4.97%	4.56%	13.30%
1×1024	3.43%	3.45%	7.71%

The 1×1024 layout has the lowest mean CV at 3.43% while the 8×8 layout has a mean CV of 78.55% and a maximum of 185.23%. Its minimum and maximum times differ strongly in several tests. The other ten layouts remain between 3.43% and 6.22% mean CV. Their maximum CV does not exceed 17.13%.

Timing becomes more stable as the workload increases. OpenMP mean CV falls from 17.64% at 512^2 to 2.02% at 4096^2. Excluding 8×8, CUDA mean CV falls from 8.33% to 1.49% over the same resolution range as larger workloads reduce the relative influence of launch, scheduling and system noise.

4.7. Minimum and maximum results

Table 9 collects the most relevant minimum and maximum values.

Table 9. Most relevant minimum and maximum values measured in the benchmark.

Legend: Times and standard deviations are in ms; CV is the standard deviation divided by the mean; Diff. px is measured against the sequential output.

Metric	Minimum	Maximum
Mean runtime	2.3491 ms CUDA 256×4 512^2, $\tau = 0.0500$	17356.1172 ms Sequential 4096^2, $\tau = 0.0005$
OpenMP speedup	$4.4051\times$ 1024^2, $\tau = 0.0005$	$5.5379\times$ 4096^2, $\tau = 0.0170$
CUDA speedup	$8.1934\times$ CUDA 8×8 512^2, $\tau = 0.0005$	$230.7196\times$ CUDA 16×16 4096^2, $\tau = 0.0170$
Timing variability	CV 0.1631% $\sigma = 26.0509$ ms Sequential, 4096^2 $\tau = 0.0500$	CV 185.2325% $\sigma = 64.4044$ ms CUDA 8×8, 512^2 $\tau = 0.0005$
CUDA Diff. px	0 px Exact match	24 px 4096^2, $\tau = 0.0005$ Every CUDA layout

The minimum mean runtime is obtained by CUDA with the 256×4 layout at 512^2 and threshold 0.0500, its output matches the sequential reference and the corresponding speedup is $109.4398\times$. The maximum mean runtime belongs to the sequential implementation at 4096^2 and threshold 0.0005.

OpenMP speedup remains between $4.4051\times$ and $5.5379\times$. The minimum appears at 1024^2 with the lowest threshold and the maximum appears at 4096^2 with threshold 0.0170.

CUDA speedup has a wider range. The minimum is produced by the 8×8 layout at 512^2 while the maximum speedup is obtained by the 16×16 layout at 4096^2 and threshold 0.0170. Its mean runtime is strongly affected by the difference between the minimum time of 2.3136 ms and the maximum time of 163.5779 ms.

The minimum CV belongs to the sequential implementation at 4096^2 while the maximum CV belongs to CUDA 8×8 at 512^2. Its standard deviation is larger than its mean runtime and produces a CV of 185.2325%.

Output consistency ranges from an exact match to 24 different pixels. The maximum difference appears at 4096^2 with threshold 0.0005 and is identical for every CUDA layout.

4.8. Cost per pixel

Table 10 uses the mean runtime across the four thresholds and CUDA uses the fixed 32×32 layout.

Table 10. Mean processing cost per pixel across the four thresholds.

Legend: Values are in ns per pixel; lower values are better.

Backend	512^2	1024^2	2048^2	4096^2
Sequential	1026.5	1008.7	1003.0	993.9
OpenMP 12T	230.1	212.3	198.1	183.4
CUDA 32×32	9.87	7.07	5.60	4.65

The sequential cost remains close to 1000 ns per pixel at every resolution. This supports the near-linear relationship between runtime and pixel count.

OpenMP cost falls from 230.1 ns per pixel at 512^2 to 183.4 ns at 4096^2. Larger images improve the amount of useful work performed between synchronization points.

CUDA cost falls from 9.87 ns per pixel to 4.65 ns per pixel, this reduction reflects the amortization of fixed launch and memory management costs.

5. Conclusions

The project implements a blob detector using the Difference of Gaussians through a sequential, OpenMP and CUDA execution paths. All the implementations use the same five Gaussian levels, four DoG maps and a common non-maximum suppression radius of six pixels allowing their outputs and performance to be compared under equivalent conditions.

The OpenMP implementation preserves the sequential result in every tested configuration and achieves a verified speedup between $4.4051\times$ and $5.5379\times$. With 12 threads this corresponds to a parallel efficiency of approximately 37% to 46%. Its advantage increases slightly with image resolution because the larger workload reduces the

relative effect of thread creation, scheduling and synchronization overhead.

CUDA provides the lowest execution times and reaches a maximum verified speedup of $192.4353\times$. Raw speedups above $220\times$ are observed at the largest resolution but these configurations have between one and 24 different binary pixels and therefore cannot be considered fully equivalent to the sequential reference. This distinction shows that the highest speedup does not necessarily correspond to the most reliable configuration.

Image resolution is the main factor affecting runtime because every stage of the scale space construction processes the complete image. The threshold has a smaller effect on execution time but it substantially changes the number and spatial distribution of detected candidates. Performance must therefore be evaluated together with output consistency rather than through runtime alone.

CUDA measurements also include initialization related work and are performed without a warm-up phase. This increases timing variability and contributes to the repeated outliers observed for the 8×8 block configuration. The CUDA layout-efficiency analysis shows that most medium and horizontal layouts remain close to the fastest CUDA result while 8×8 and 1×1024 are less robust.

Overall the results show that OpenMP provides a stable CPU acceleration while preserving output consistency. CUDA offers substantially greater performances at the cost of higher sensitivity to launch configuration, initialization overhead and small output differences. The most appropriate implementation therefore depends on whether the priority is exact reproducibility, predictable CPU scaling or maximum execution speed.

6. Future work

- *CUDA timing*: Add warm-up runs and separate allocation, transfer and kernel times.
- *Memory reuse*: Preserve CPU workspaces and CUDA buffers across repetitions.
- *For collapse*: Repeat the tests while using the collapse OpenMP instruction to better parallelize the loops.
- *Image set*: Repeat the benchmark with native images containing different textures and feature densities.